Object-Oriented Software Engineering

Using UML, Patterns, and Java

Object Design II: Introduction to Design Patterns



Bernd Bruegge

Applied Software Engineering

Technische Universitaet Muenchen

- During Object Modeling we do many transformations and changes to the object model
- It is important to make sure the object design model stays simple!

- Design patterns helps keep system models simple.

Finding Objects

- The hardest problems in object-oriented system development are:
  - Identifying objects
  - Decomposing the system into objects
- Requirements Analysis focuses on application domain:
  - Object identification
- System Design addresses both, application and implementation domain:
  - Subsystem Identification
- Object Design focuses on implementation domain:
  - Additional solution objects

Techniques for Finding Objects

- Requirements Analysis
  - Start with Use Cases. Identify participating objects
  - Textual analysis of flow of events (find nouns, verbs, ...)
  - Extract application domain objects by interviewing client (application domain knowledge)
  - Find objects by using general knowledge
  - Extract objects from Use Case scenarios (dynamic model)
- System Design
  - Subsystem decomposition
  - Try to identify layers and partitions
- Object Design
  - Find additional objects by applying implementation domain knowledge

Another Source for Finding Objects : Design Patterns

- What are Design Patterns?
  - The recurring aspects of designs are called *design patterns* [Gamma et al 1995].
  - A *pattern* is the outline of a reusable solution to a general problem encountered in a particular context.
  - It describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same twice. Many of them have been systematically documented for all software developers to use.

Studying patterns is an effective way to learn from the experience of others

What is common between these definitions?

- Definition Software System
  - A software system consists of subsystems which are either other subsystems or collection of classes

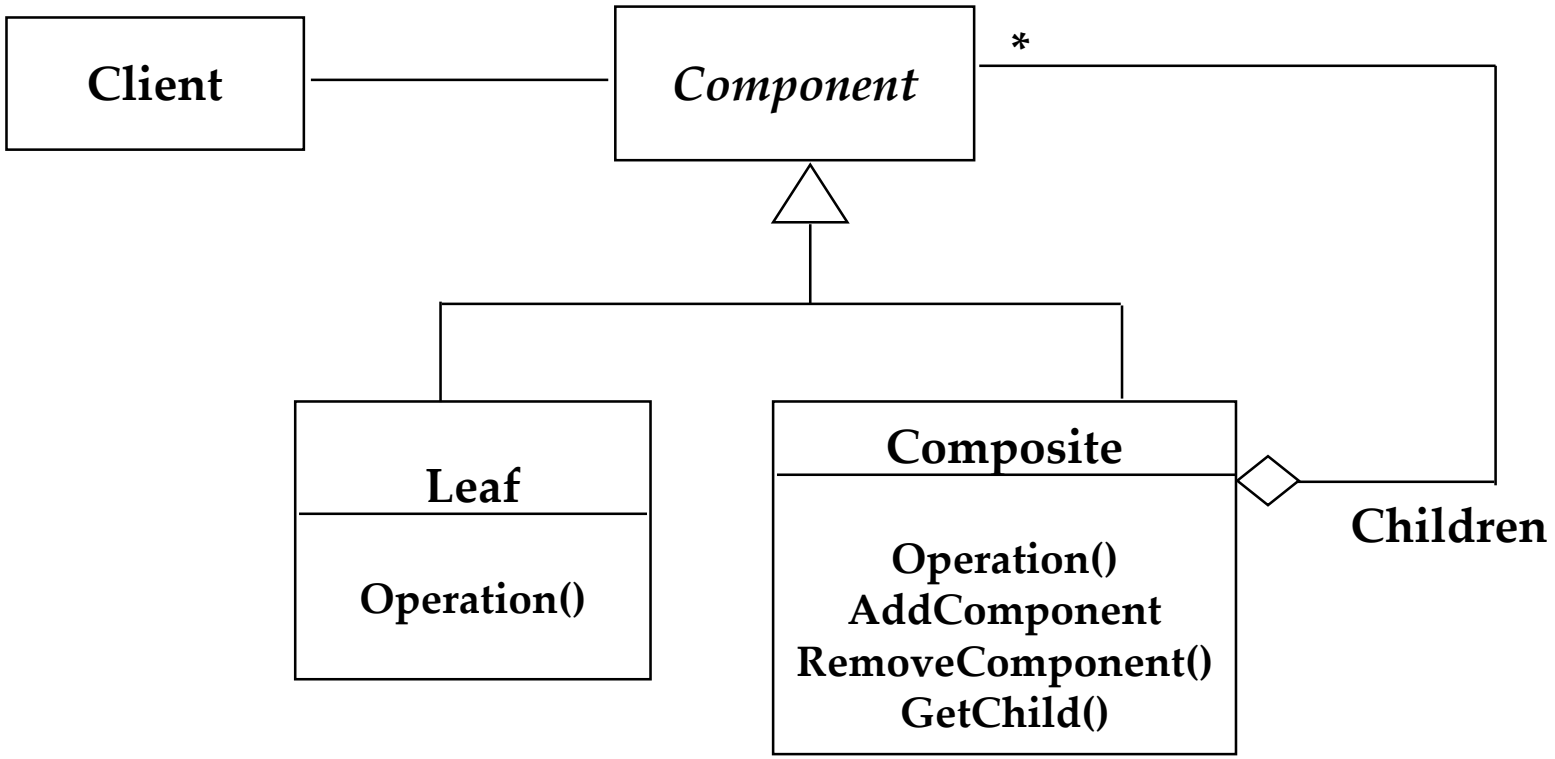
- Definition Software Lifecycle
  - A software lifecycle consists of a set of development activities which are either other actitivies or collection of tasks.

What is common between these definitions?

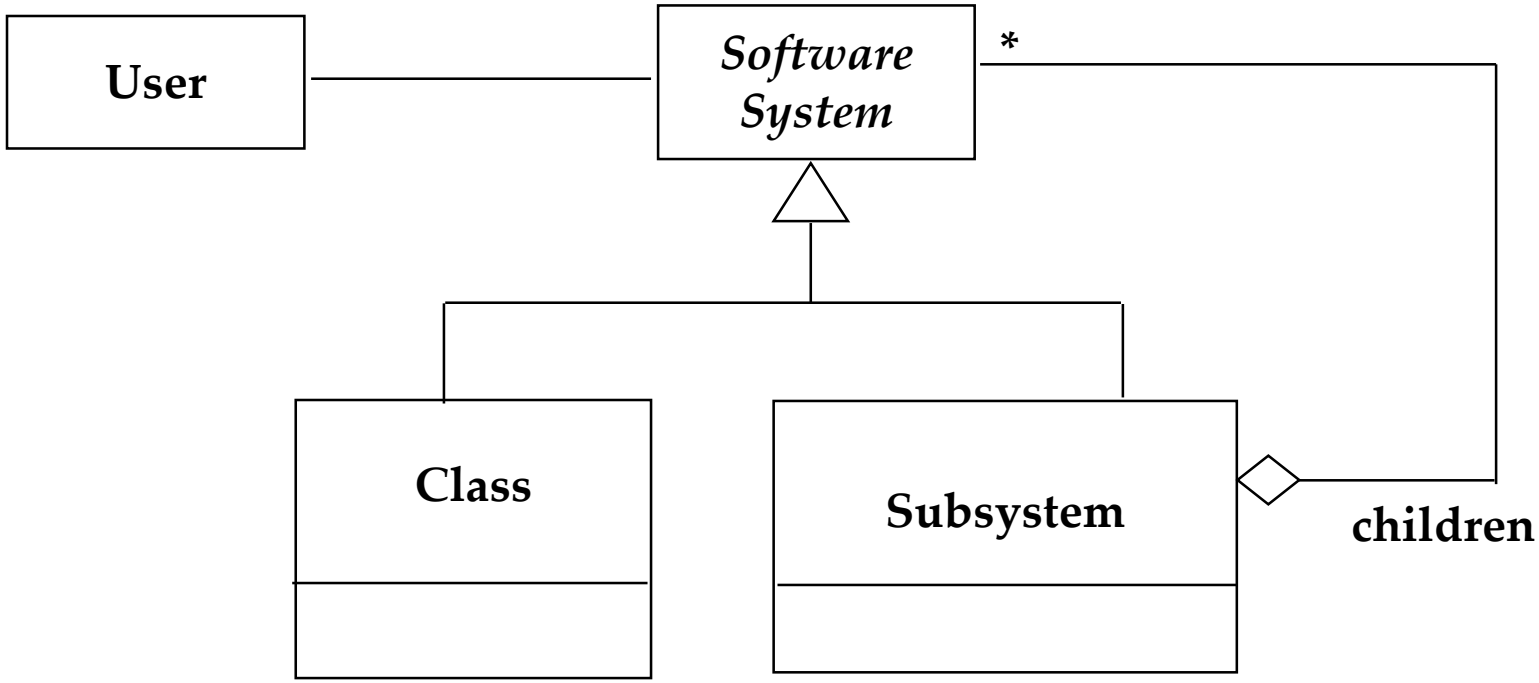
- Definition Software System
  - A software system consists of subsystems which are either other subsystems or collection of classes
  - Composite: Subsystem (A software system consists of subsystems which consists of subsystems, which consists of subsystems, which...)
  - Leaf node: Class
- Definition Software Lifecycle
  - A software lifecycle consists of a set of development activities which are either other actitivies or collection of tasks
  - Composite: Activity (A software lifecycle consists of activities which consist of activities, which consist of activities, which....)
  - Leaf node: Task.

Introducing the Composite Pattern

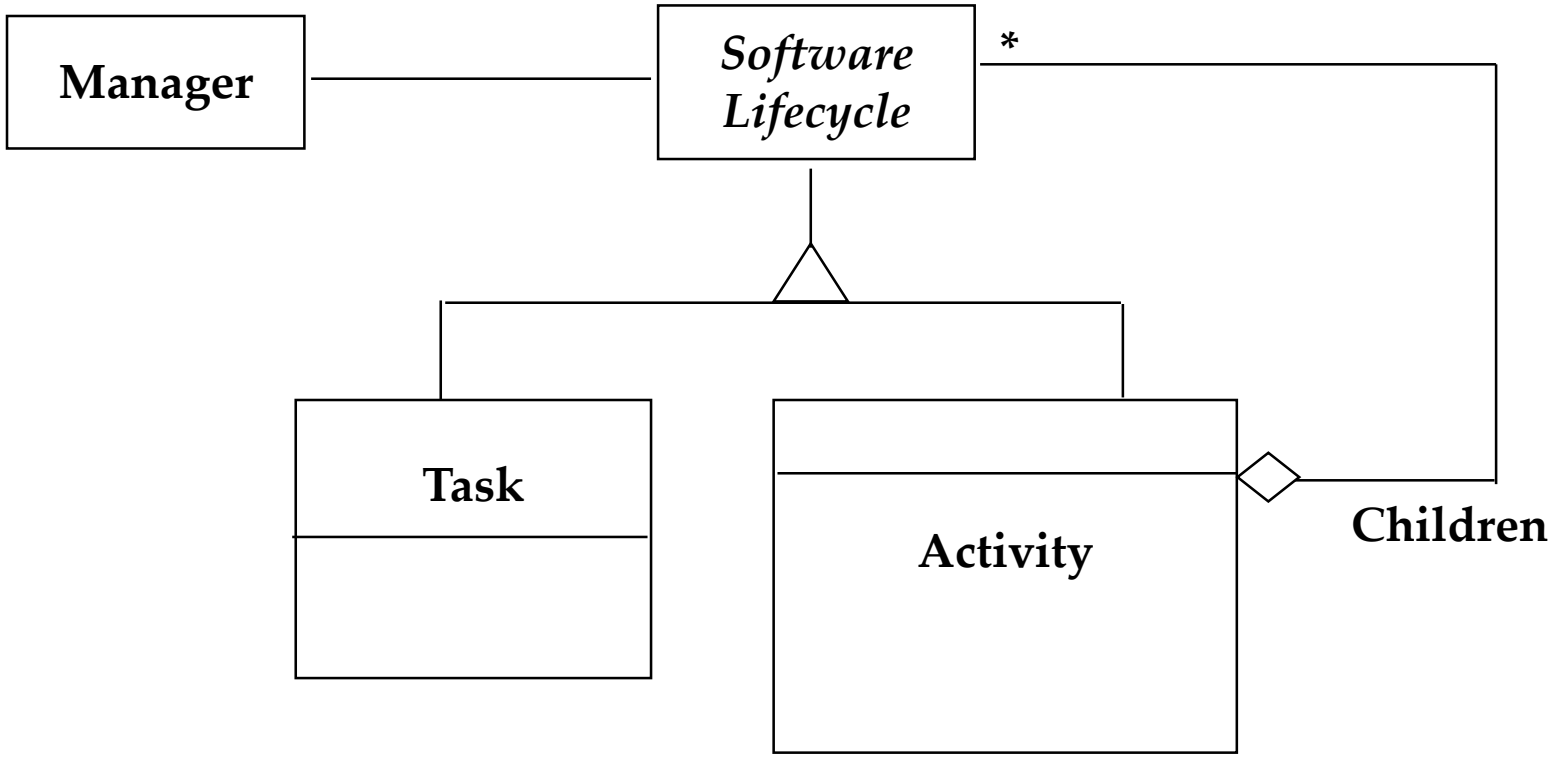
- Models tree structures that represent part-whole hierarchies with arbitrary depth and width.
- The Composite Pattern lets client treat individual objects and compositions of these objects uniformly



Modeling a Software System with a Composite Pattern

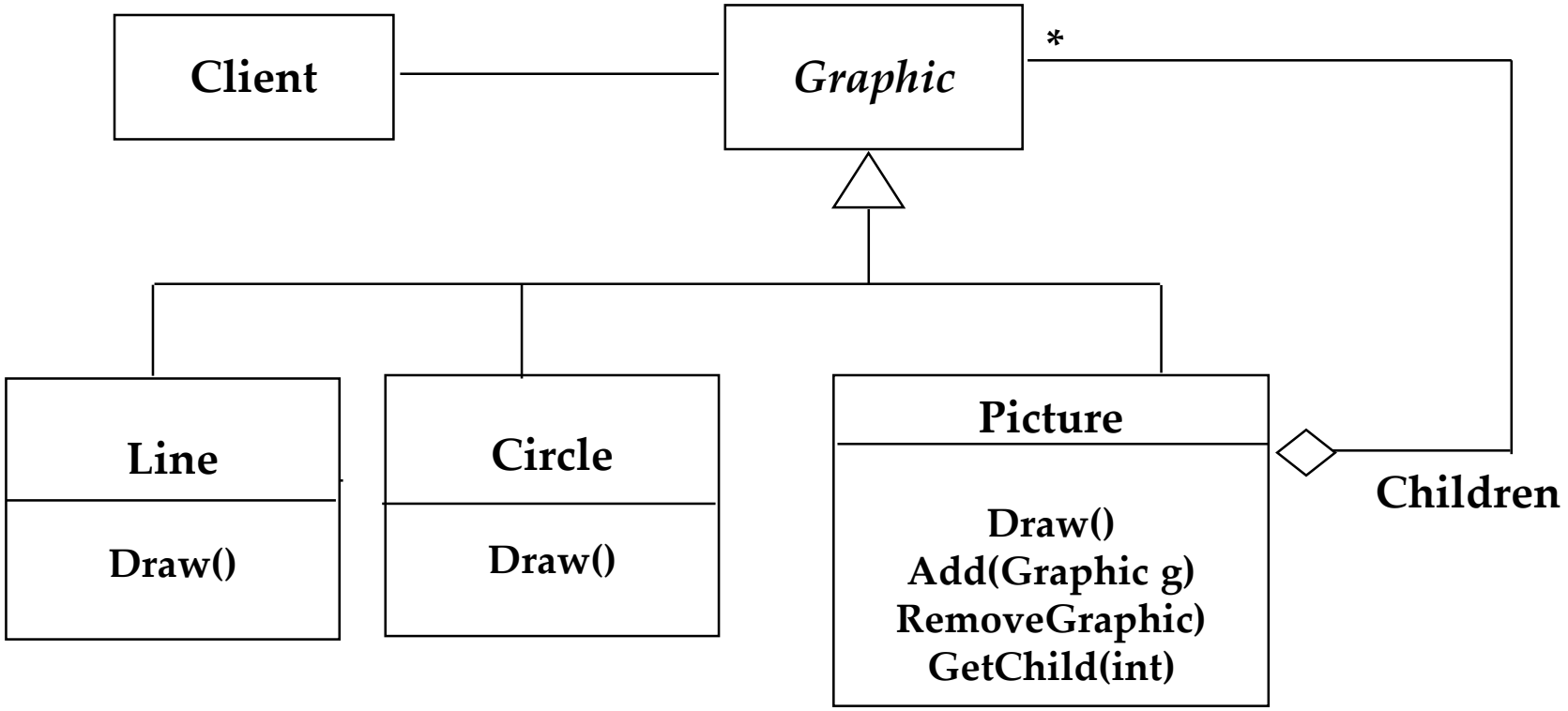


Modeling the Software Lifecycle with a Composite Pattern



Graphic Applications also use Composite Patterns

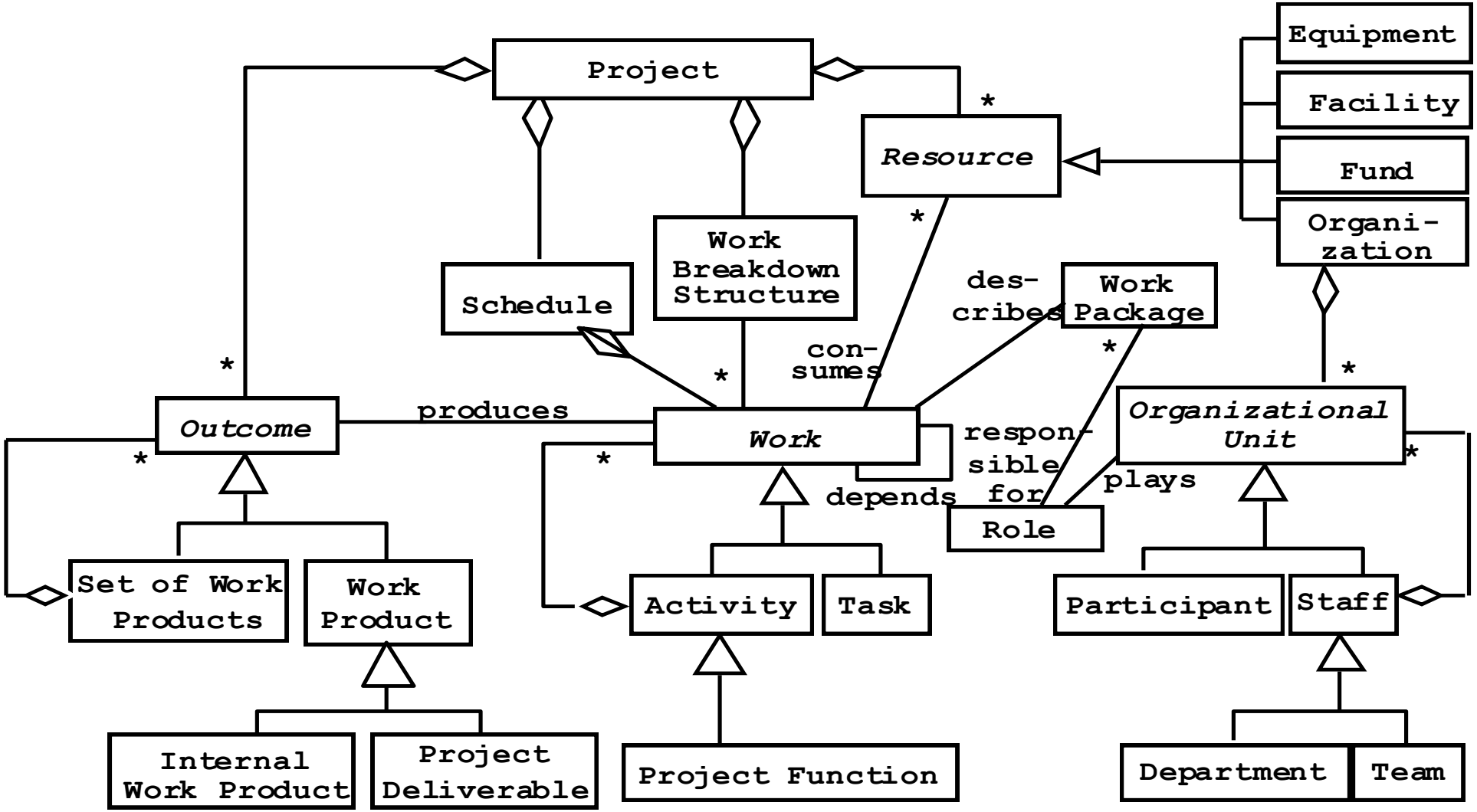
- The *Graphic* Class represents both primitives (Line, Circle) and their containers (Picture)



Reducing the Complexity of Models

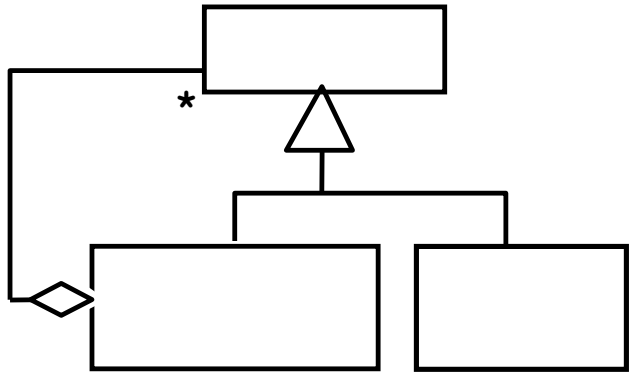
- Modeling is about communication
  - To a human being as well as to a tool
- To communicate a complex model to a human being we use navigation and reduction of complexity
  - Navigate through the model so the user can follow it
- Start with a very simple model
  - Identify the key abstractions
  - Then decorate the model with additional classes
- To reduce the complexity of the model further
  - Look for inheritance (taxonomies)
    - If the model is too complex, taxonomies should be placed in separate UML packages
  - Then identify or introduce patterns in the model
    - Make sure to use the name of the patterns.

Example: Modeling a Project



How to reduce the Complexity of the Model

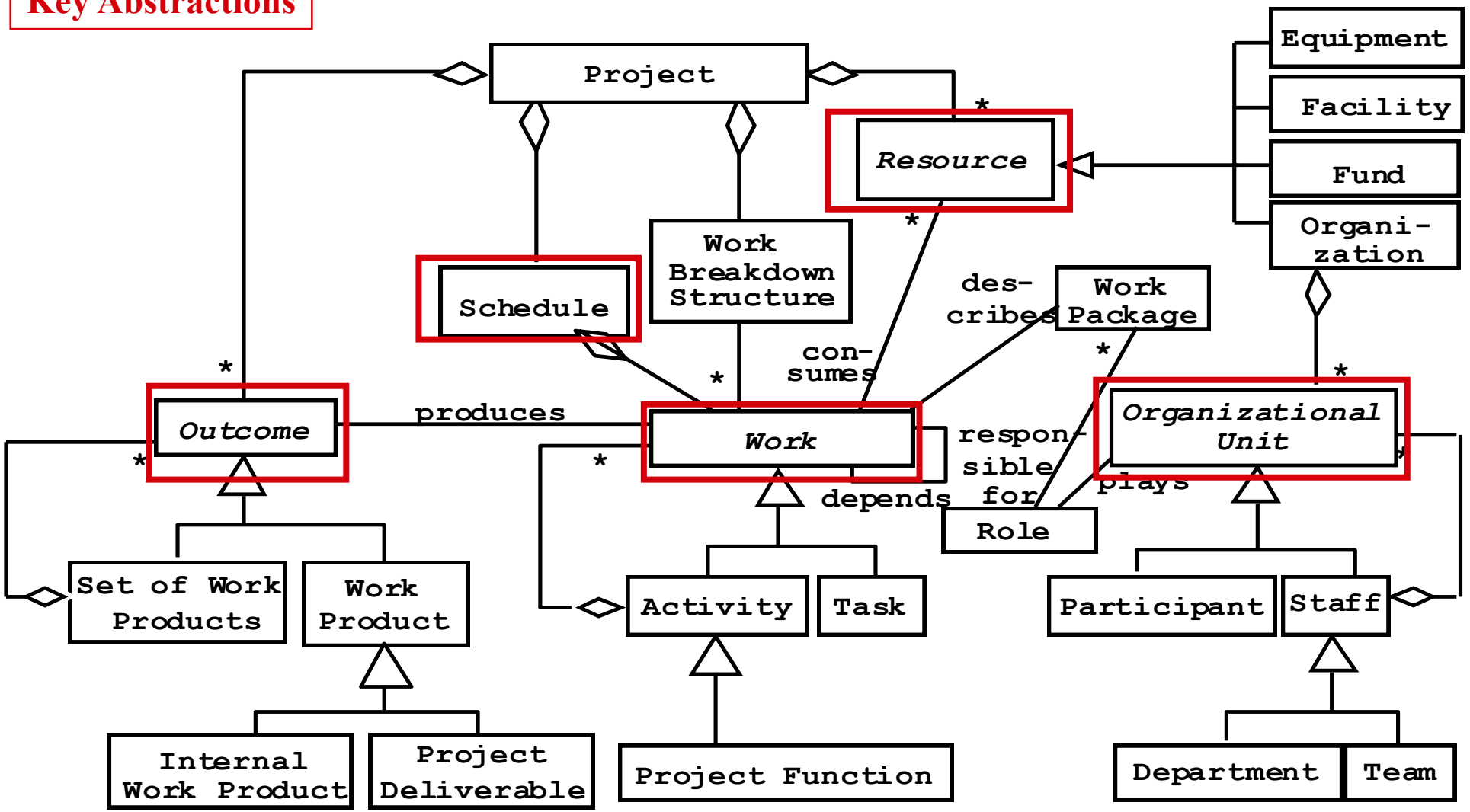
1. Look for the key abstractions
   - Project, Outcome, Schedule, Work, Resource
2. Identify patterns: For example, the project model has 3 composite patterns



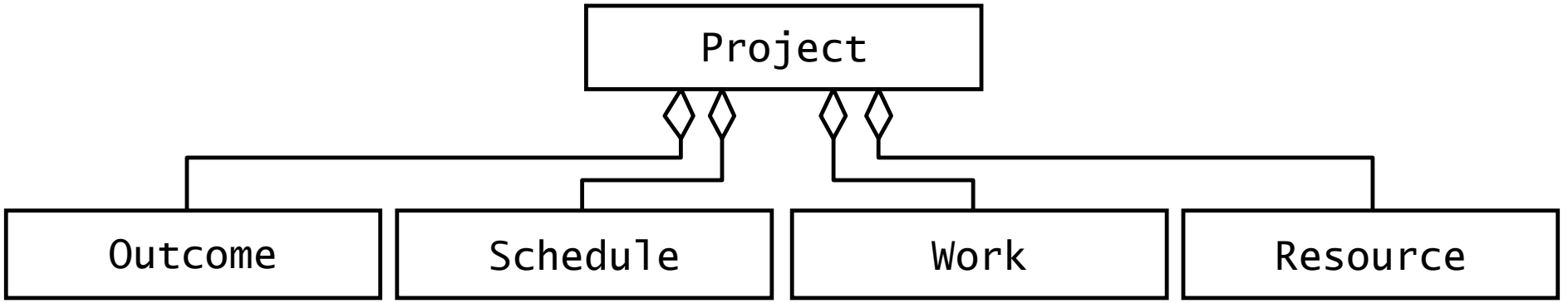
3. Find all the application domain taxonomies
   - Start with the taxonomies for the key abstractions
4. Key abstractions, patterns and/or taxonomies are candidates for separate UML packages.

1. Find the Key Abstractions in the Model

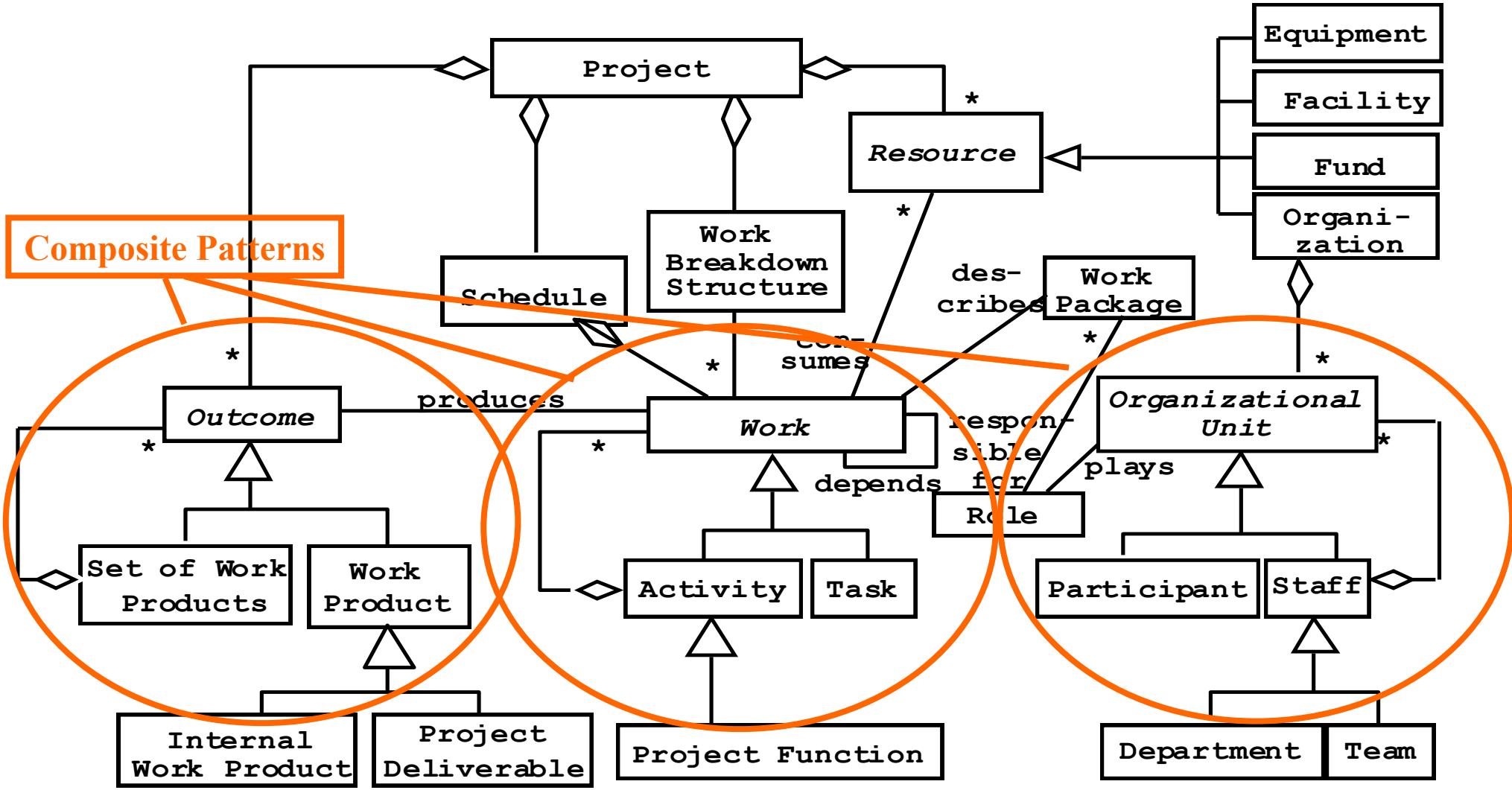
Key Abstractions



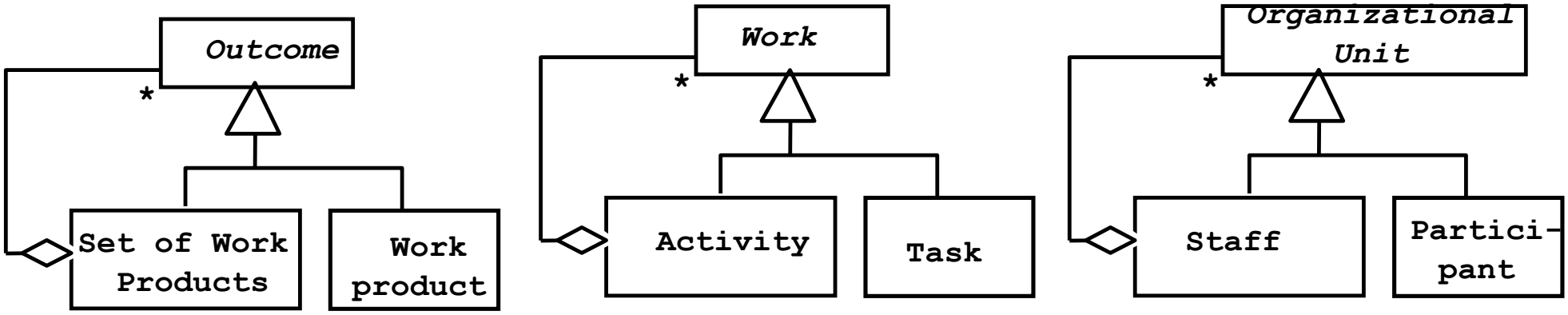
Key Abstractions in the Project Model



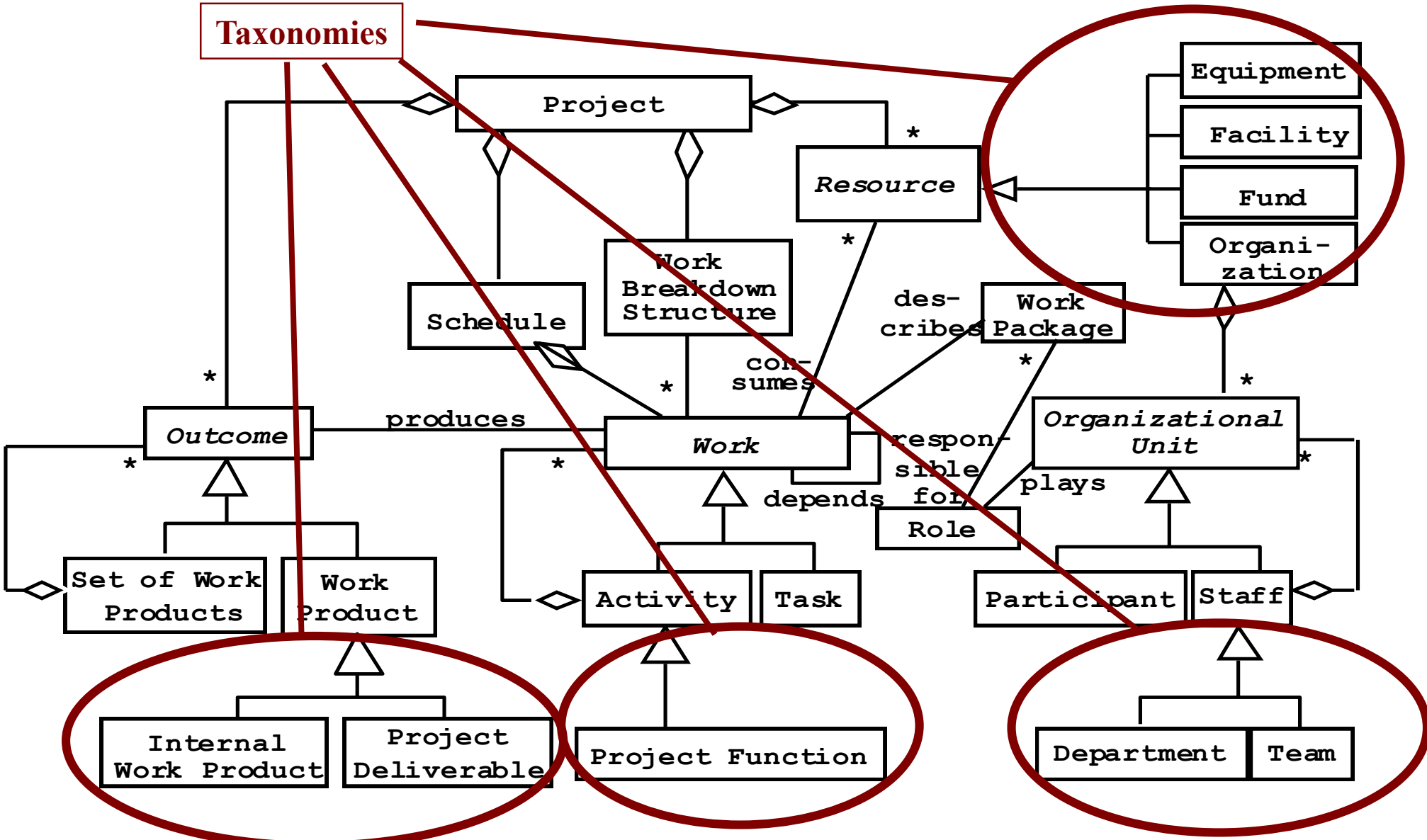
2. Find Patterns used in the Model



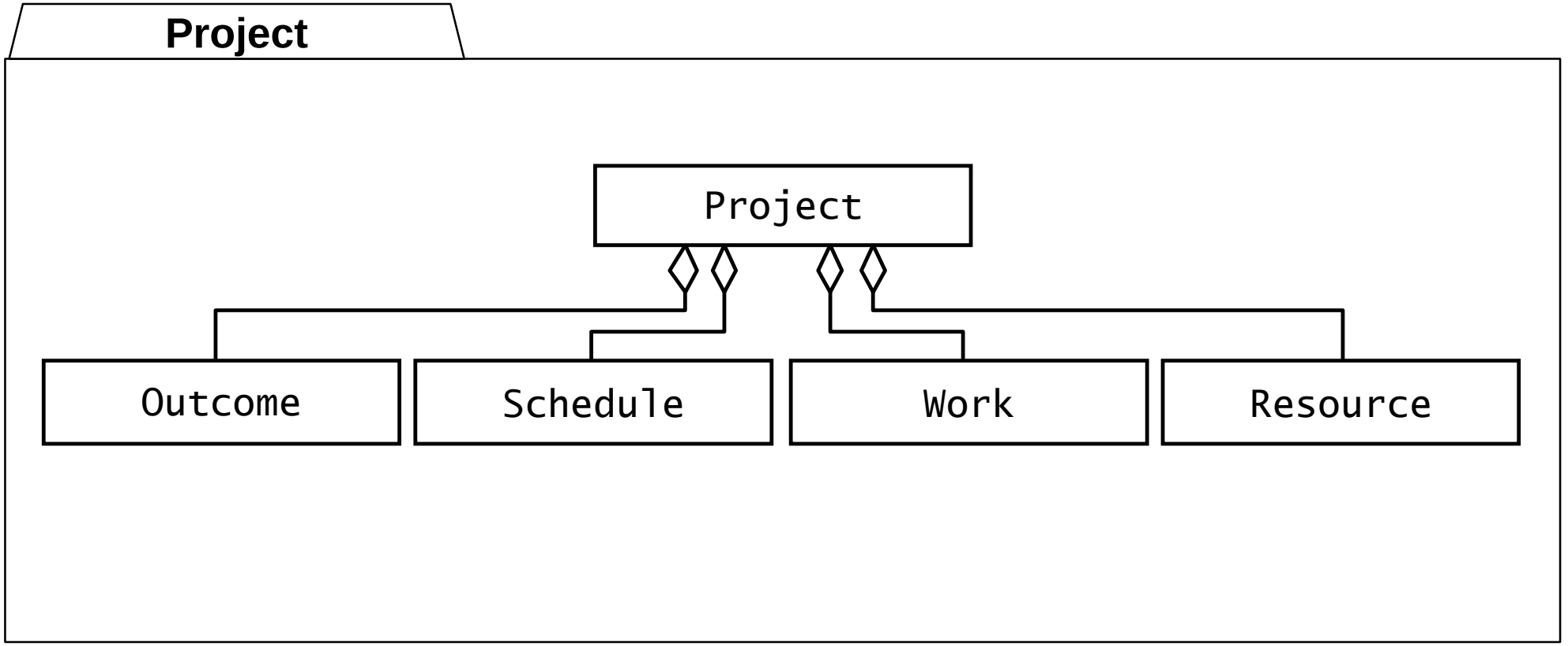
Composite Patterns in the Project Model



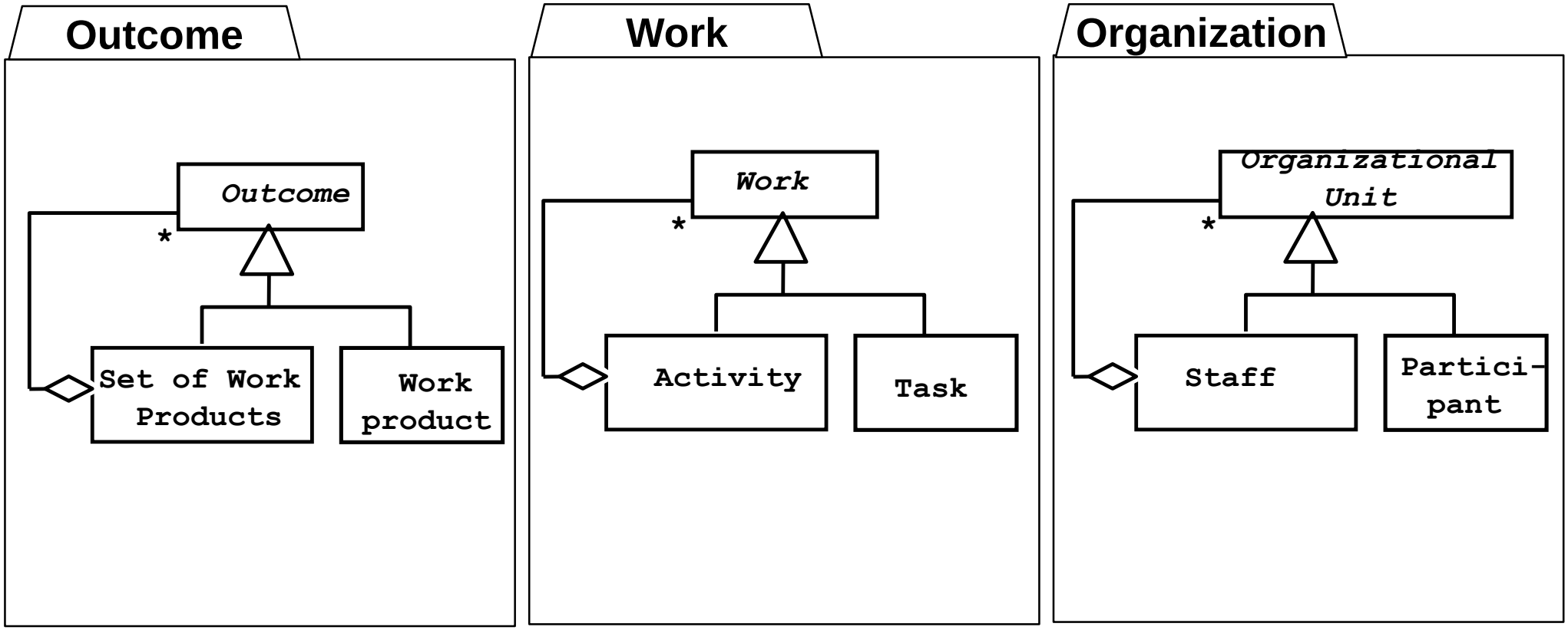
3. Find the Taxonomies used in the Model



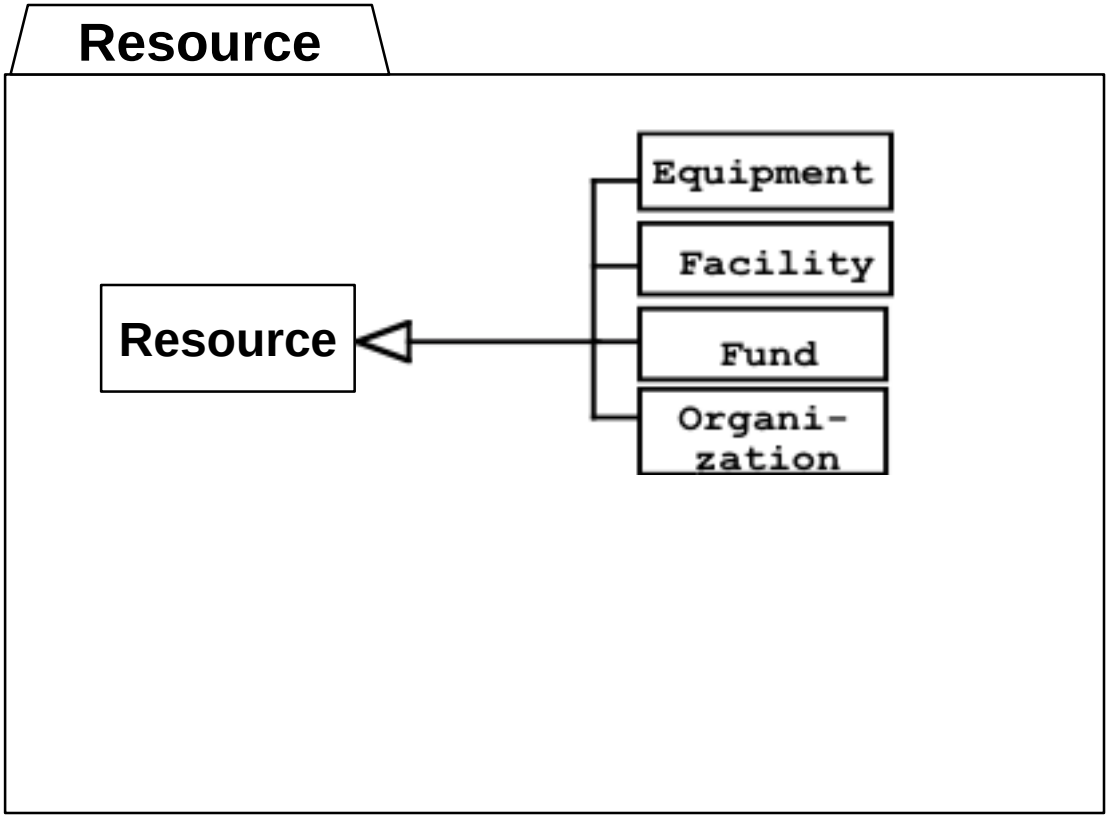
Step 4: Package based on Key Abstractions



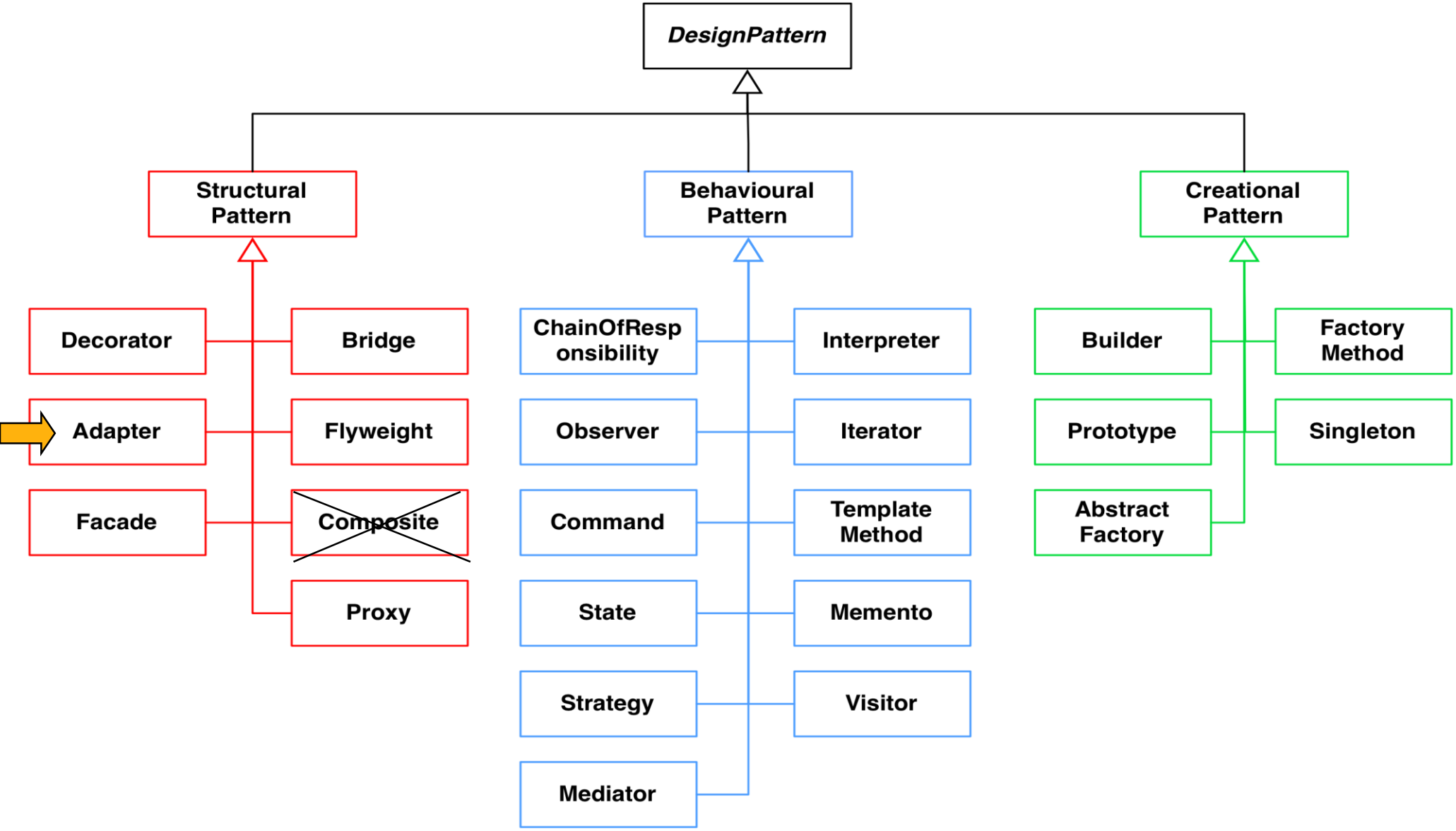
Step 4 continued: Additional Packages based on Patterns



Step 4 continued:
Additional UML Packages based on Taxonomies

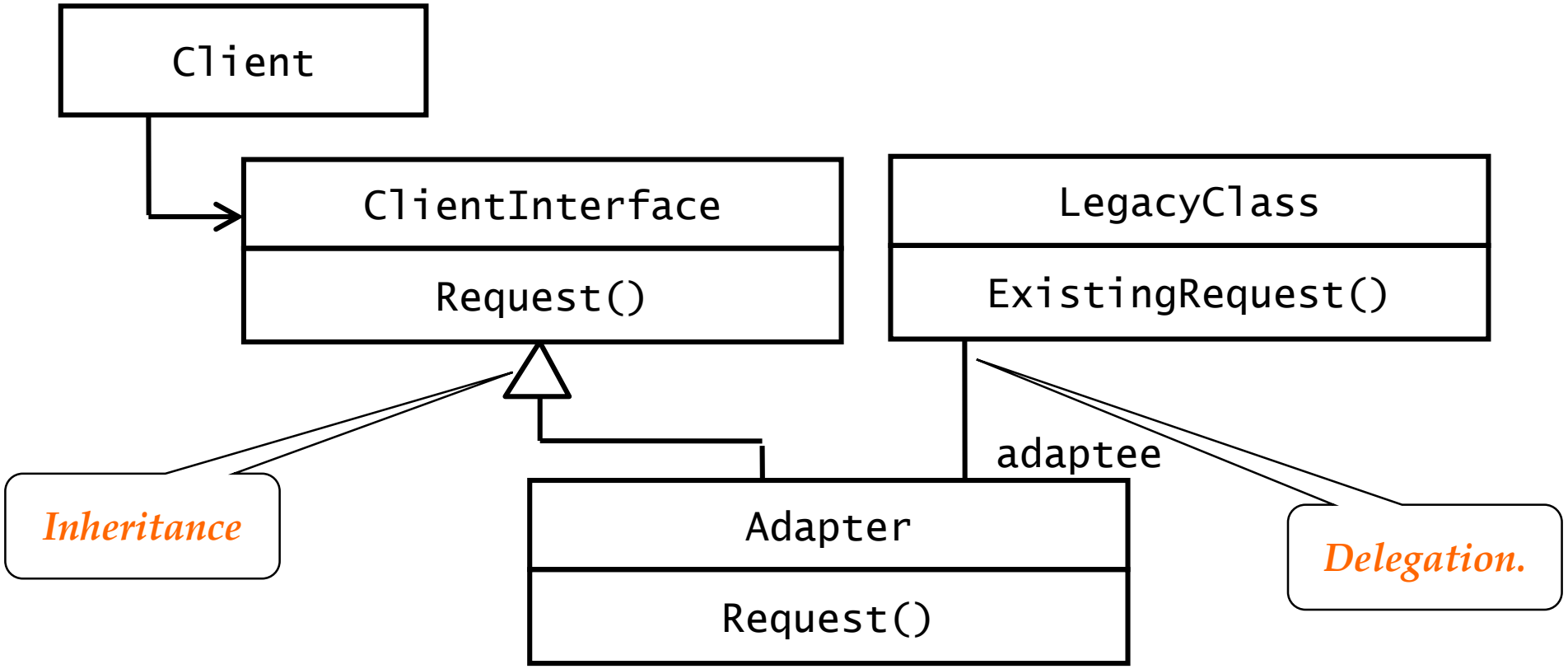


Taxonomy of Design Patterns (23 Patterns)



Many design patterns use a combination of inheritance and delegation

Adapter Pattern



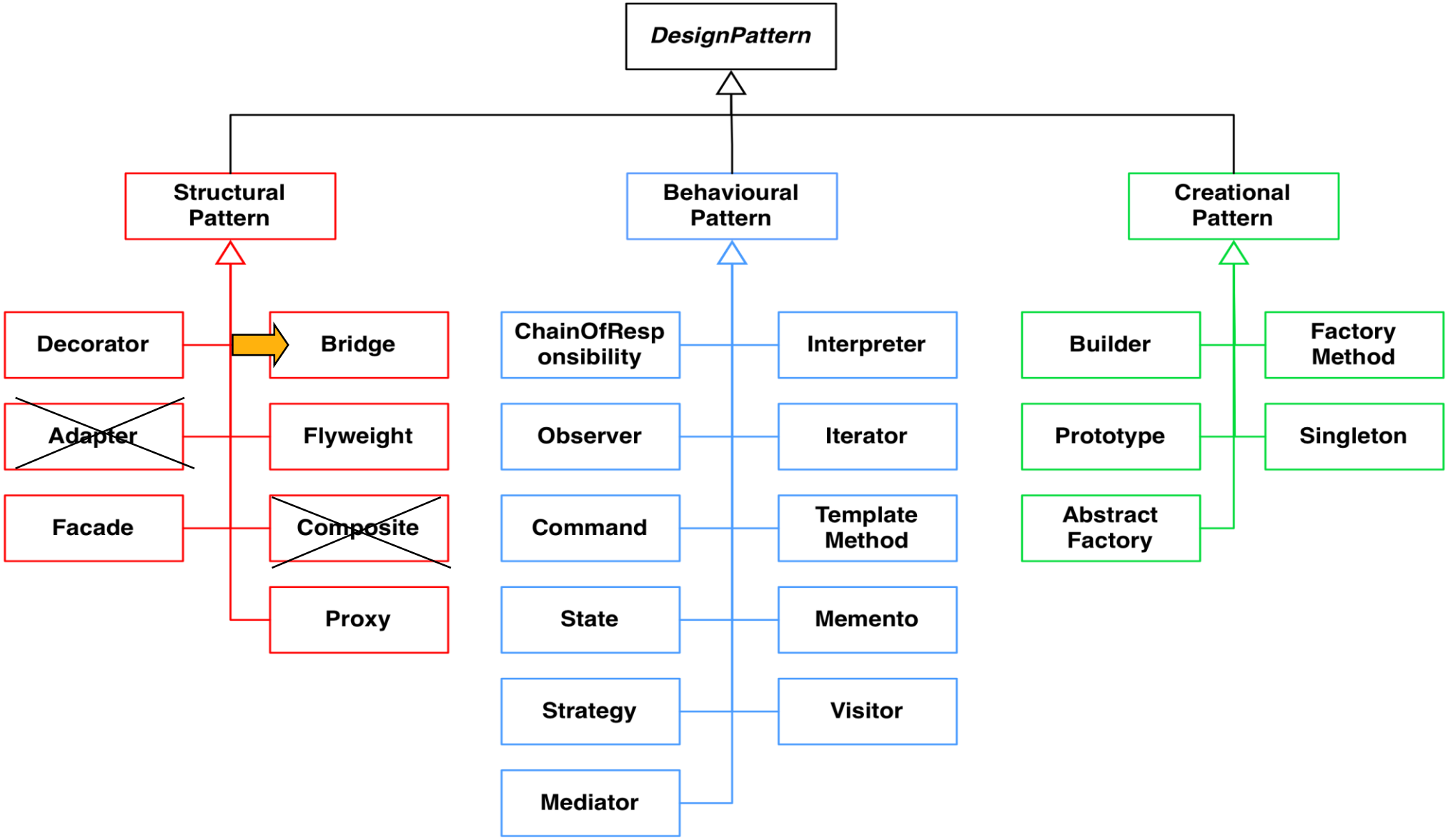
The adapter pattern uses inheritance as well as delegation:

- *Interface inheritance: Adapter inherits Request() from ClientInterface*
- *Delegation: Binds LegacyClass to the Adapter.*

Adapter Pattern

- The adapter pattern lets classes work together that couldn't otherwise because of incompatible interfaces
  - "Convert the interface of a class into another interface expected by a client class."
  - Used to provide a new interface to existing legacy components (Interface engineering, reengineering).
- Two adapter patterns:
  - Class adapter:
    - Uses multiple inheritance to adapt one interface to another
  - Object adapter:
    - Uses single inheritance and delegation
- Object adapters are much more frequent.
- We cover only object adapters (and call them adapters).

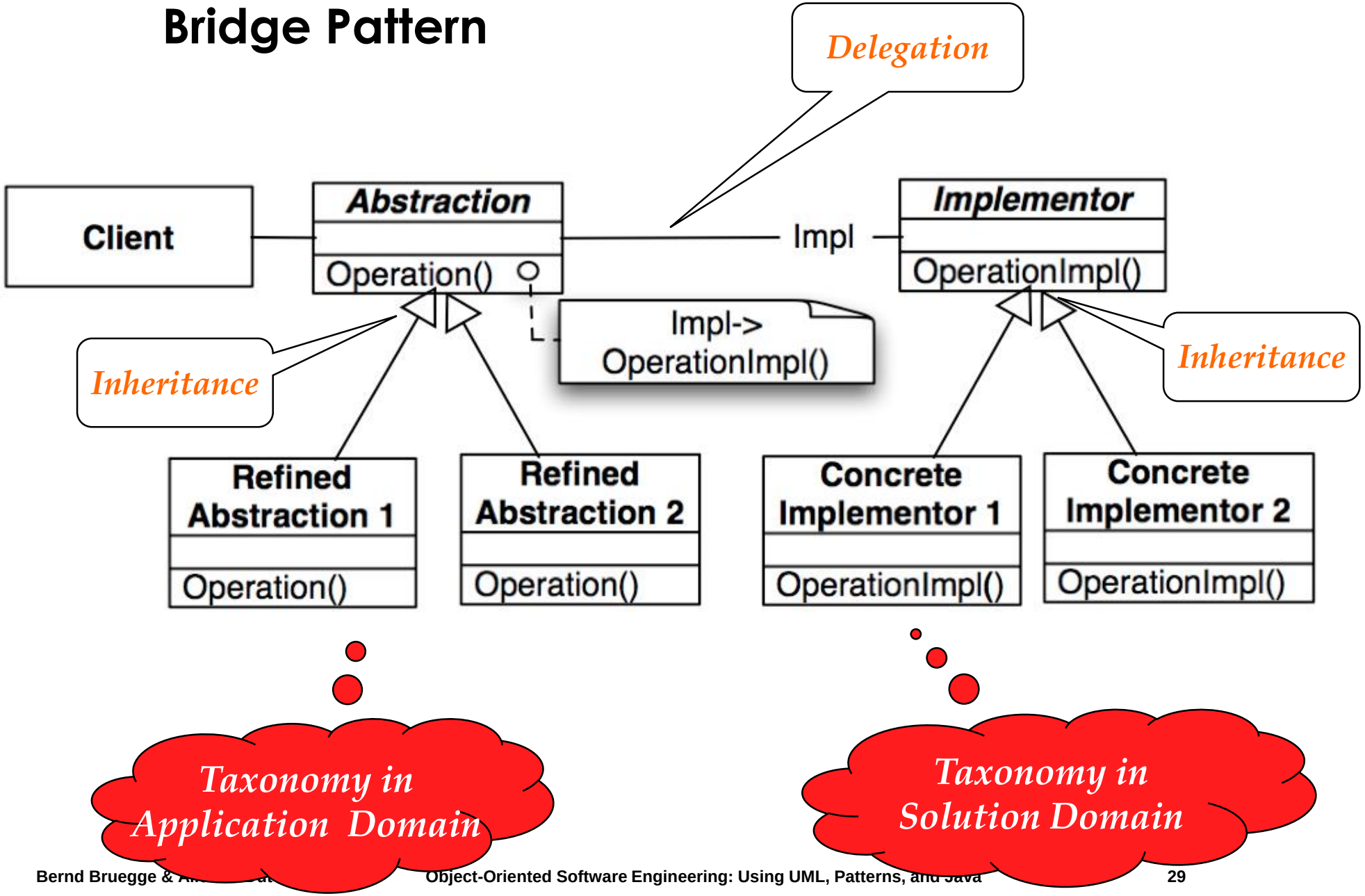
Taxonomy of Design Patterns



Bridge Pattern

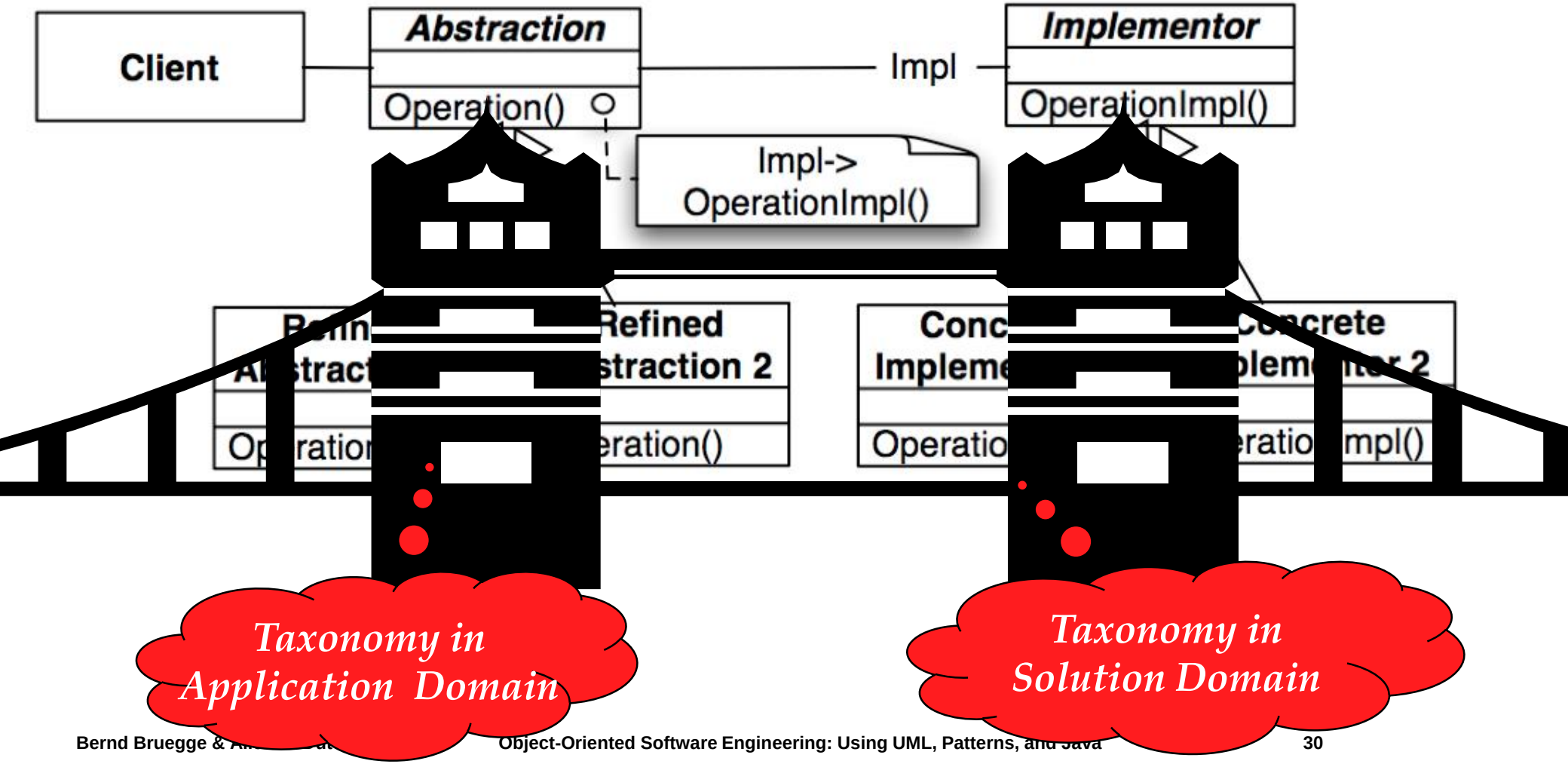
- Use a bridge to "decouple an abstraction from its implementation so that the two can vary independently"

Bridge Pattern



Why the Name Bridge Pattern?

It provides a bridge between the Abstraction (in the application domain) and the Implementor (in the solution domain)

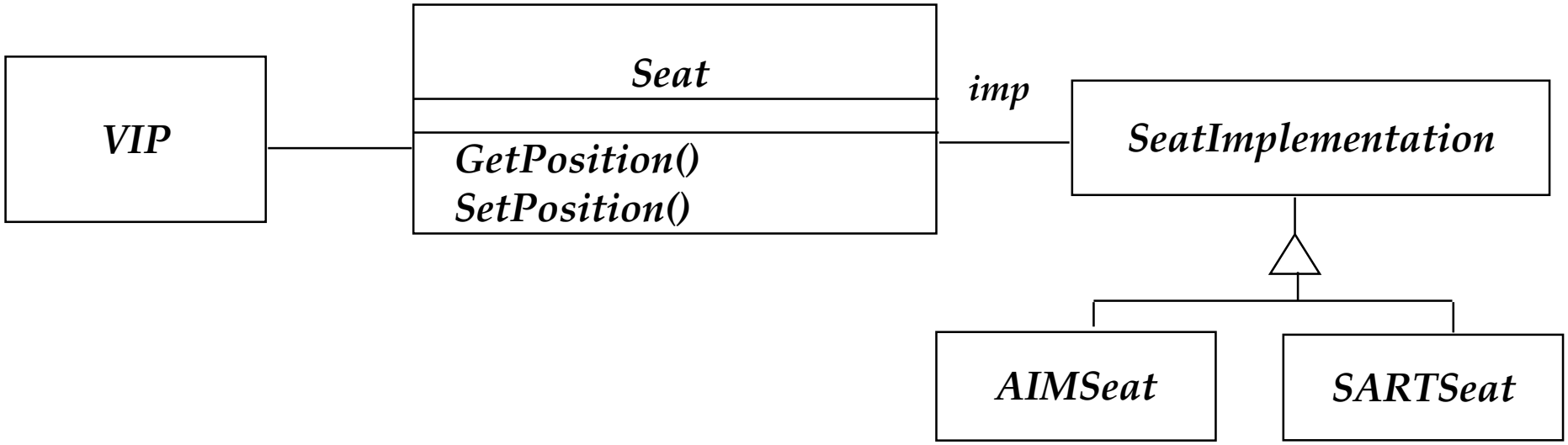


Motivation for the Bridge Pattern

- Decouples an abstraction from its implementation so that the two can vary independently
- This allows to bind one from many different implementations of an interface to a client dynamically
- Design decision that can be realized any time during the runtime of the system
  - However, usually the binding occurs at start up time of the system (e.g. in the constructor of the interface class)

Using a Bridge

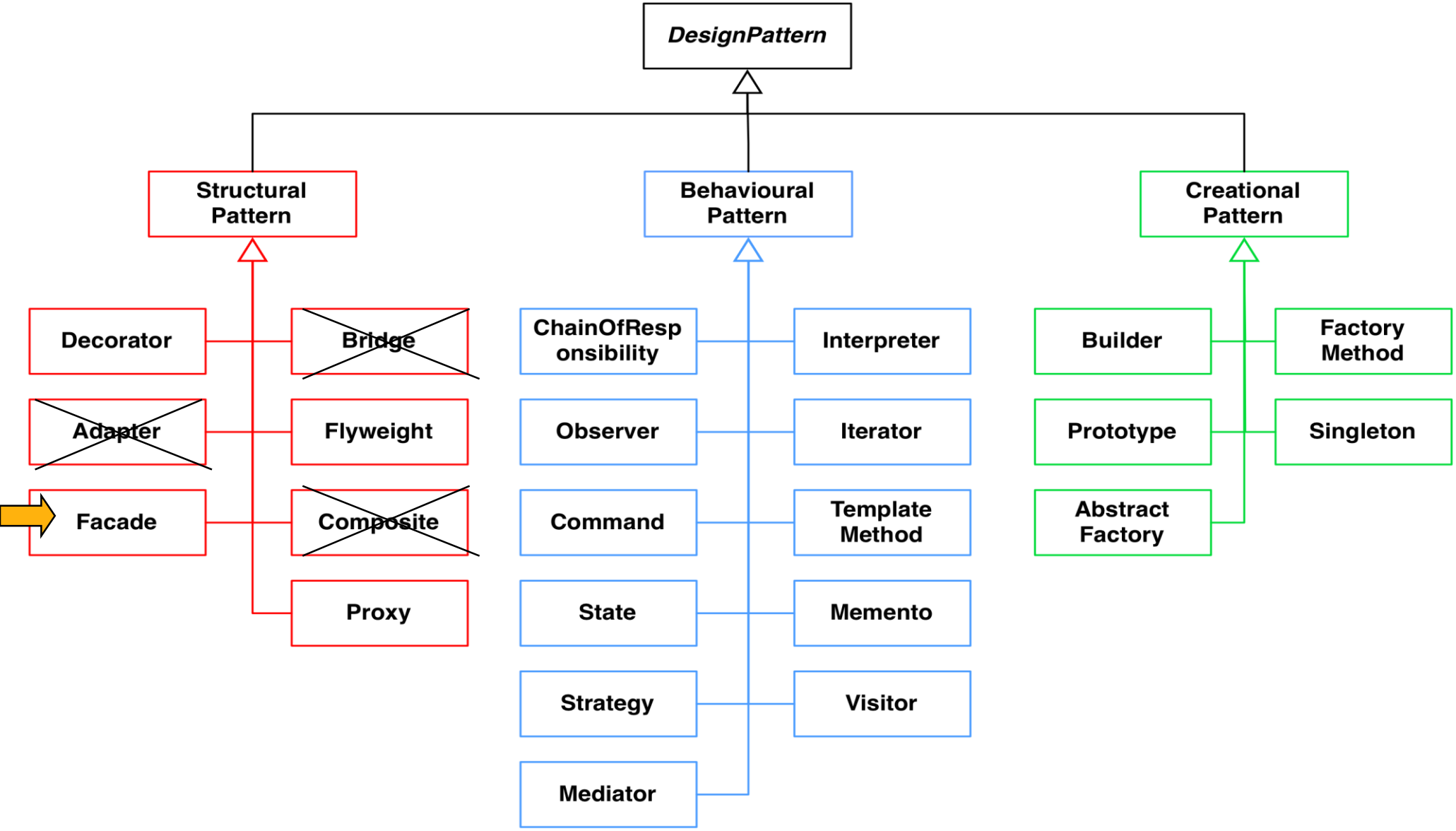
- The bridge pattern can be used to provide multiple implementations under the same interface
  - Example: Interface to a component that is incomplete, not yet known or unavailable during testing
    - GetPosition() is needed by VIP, but the class Seat is only available by two simulations (AIMSeat and SARTSeat). To switch between these, the bridge pattern can be used:



Adapter vs Bridge

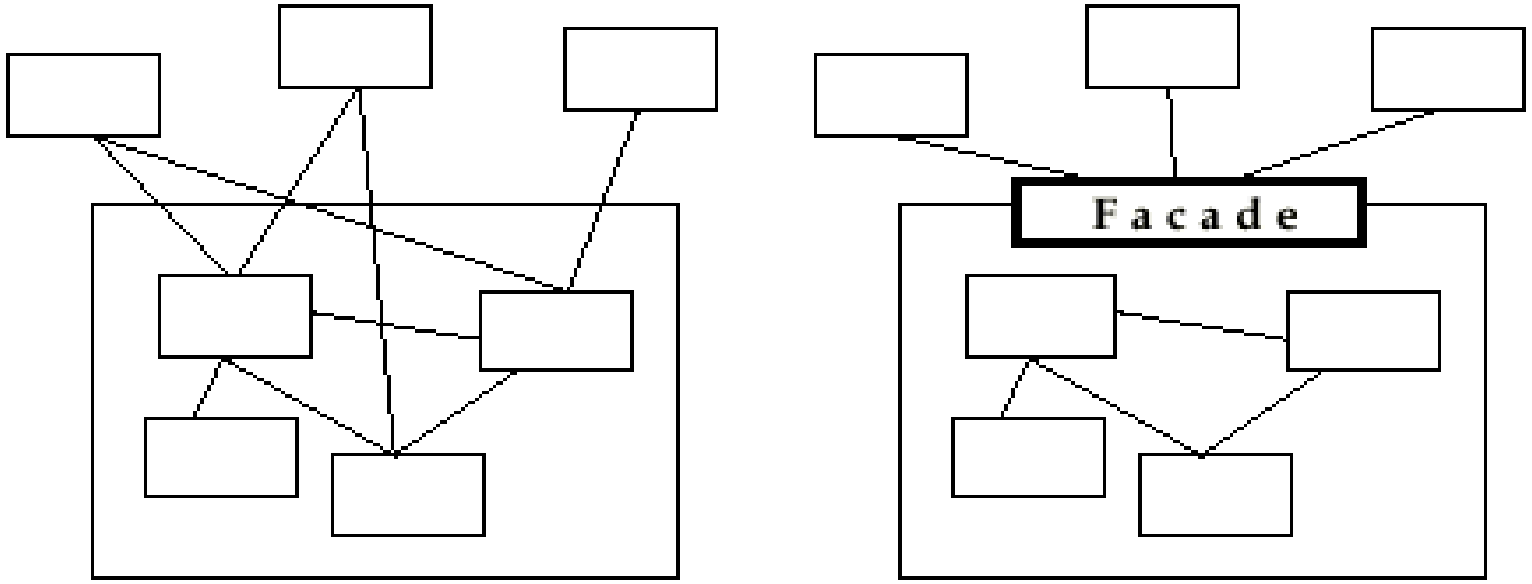
- Similarities:
  - Both are used to hide the details of the underlying implementation.
- Difference:
  - The adapter pattern is geared towards making unrelated components work together
    - Applied to systems after they're designed (reengineering, interface engineering).
    - "Inheritance followed by delegation"
  - A bridge, on the other hand, is used up-front in a design to let abstractions and implementations vary independently.
    - "Delegation followed by inheritance"

Taxonomy of Design Patterns (23 Patterns)



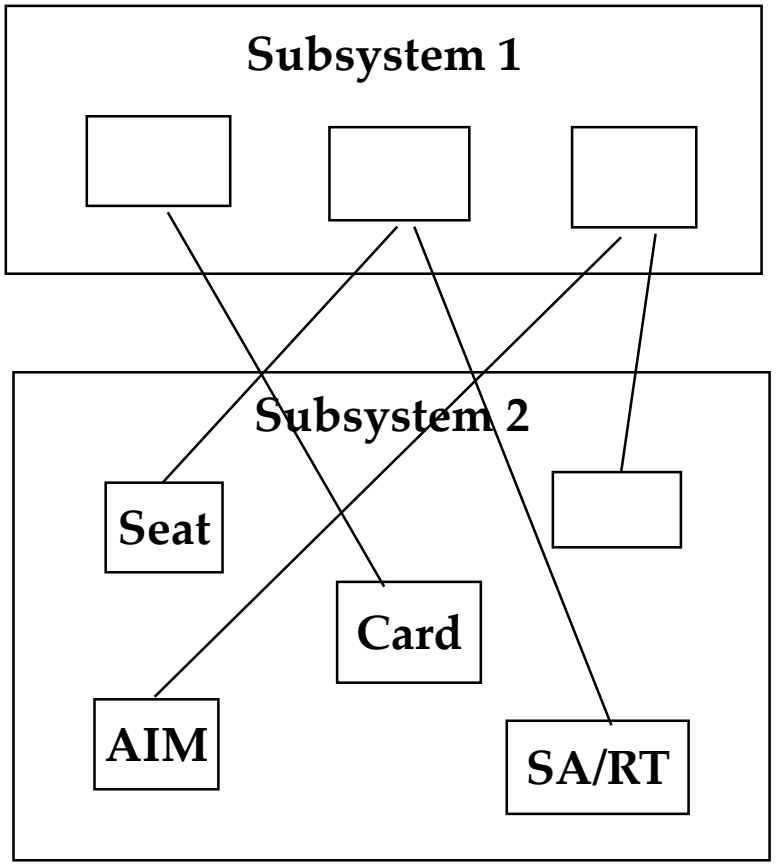
Facade Pattern

- Provides a unified interface to a set of objects in a subsystem.
- A facade defines a higher-level interface that makes the subsystem easier to use (i.e. it abstracts out the gory details)
-



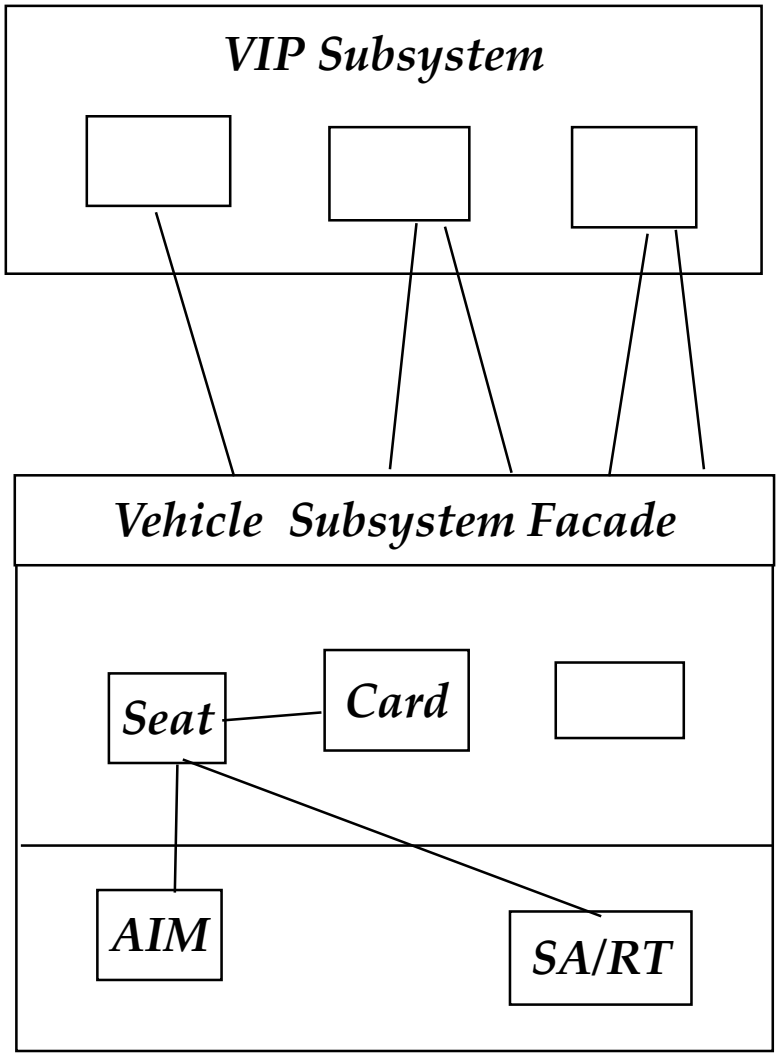
Design Example

- Subsystem 1 can look into the Subsystem 2 (vehicle subsystem) and call on any component or class operation at will.
- This is "Ravioli Design"
- Why is this good?
  - Efficiency
- Why is this bad?
  - Can't expect the caller to understand how the subsystem works or the complex relationships within the subsystem.
  - We can be assured that the subsystem will be misused, leading to non-portable code



Realizing an Opaque Architecture with a Facade

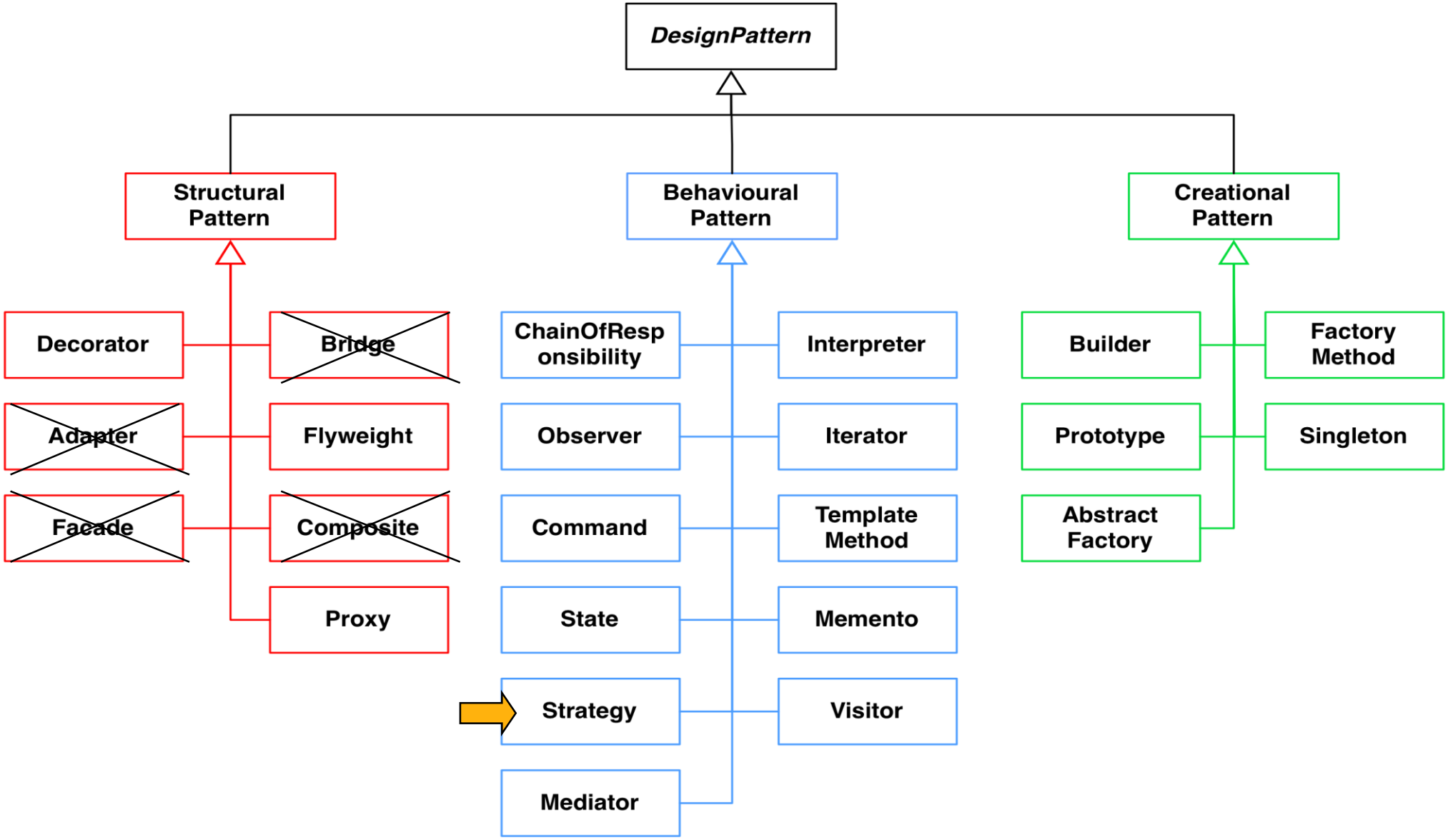
- The Vehicle Subsystem decides exactly how it is accessed
- No need to worry about misuse by callers
- We need to write only stubs for each of the public methods in the façade.



Subsystem Design with Façade, Adapter, Bridge

- The ideal structure of a subsystem consists of
  - an interface object
  - a set of application domain objects (entity objects) modeling real entities or existing systems
    - Some of the application domain objects are interfaces to existing systems
  - one or more control objects
- We can use design patterns to realize this subsystem structure
- Realization of the Interface Object: Facade
  - Provides the interface to the subsystem
- Interface to existing systems: Adapter or Bridge
  - Provides the interface to existing system (legacy system)
  - The existing system is not necessarily object-oriented!

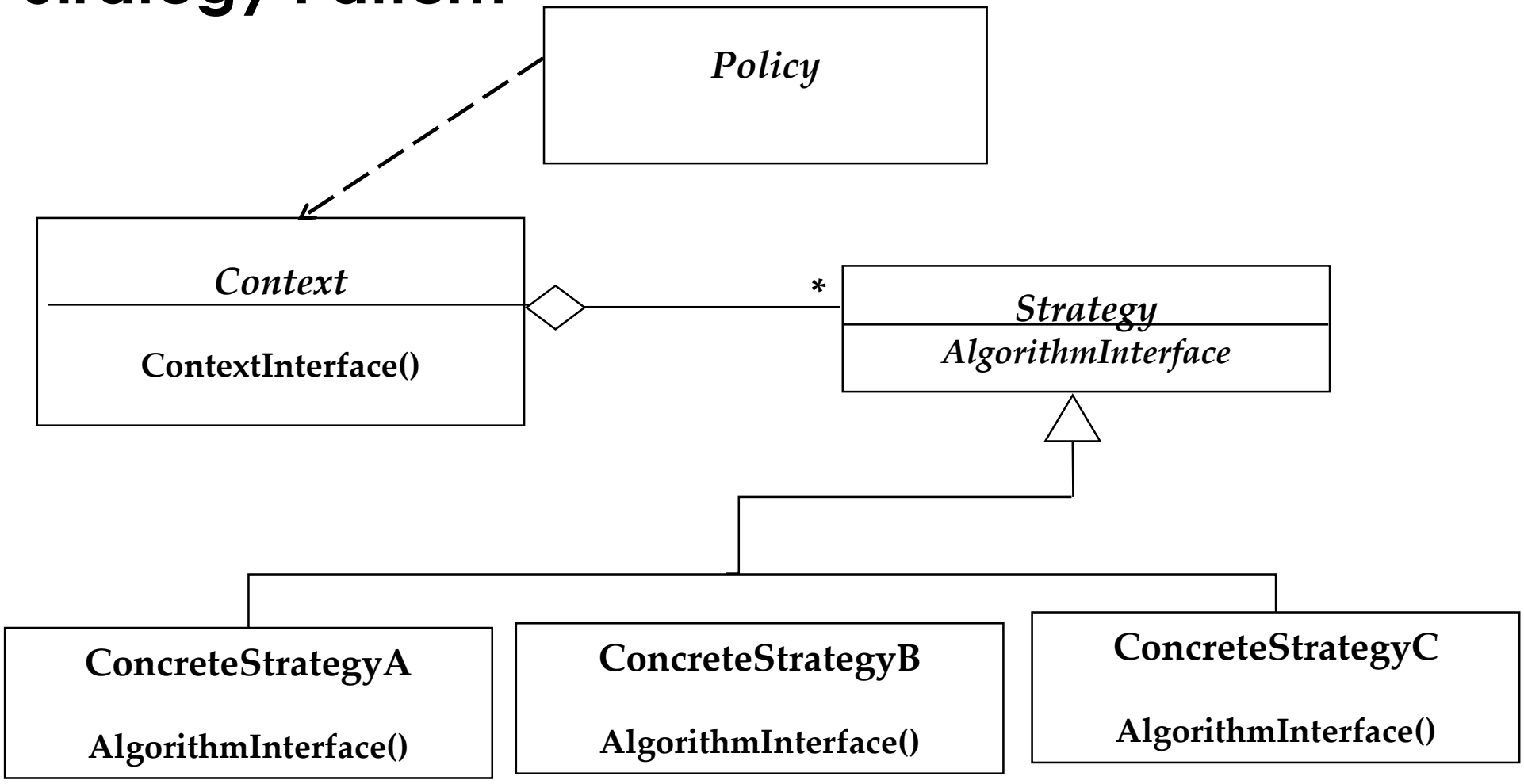
Taxonomy of Design Pattern



Strategy Pattern

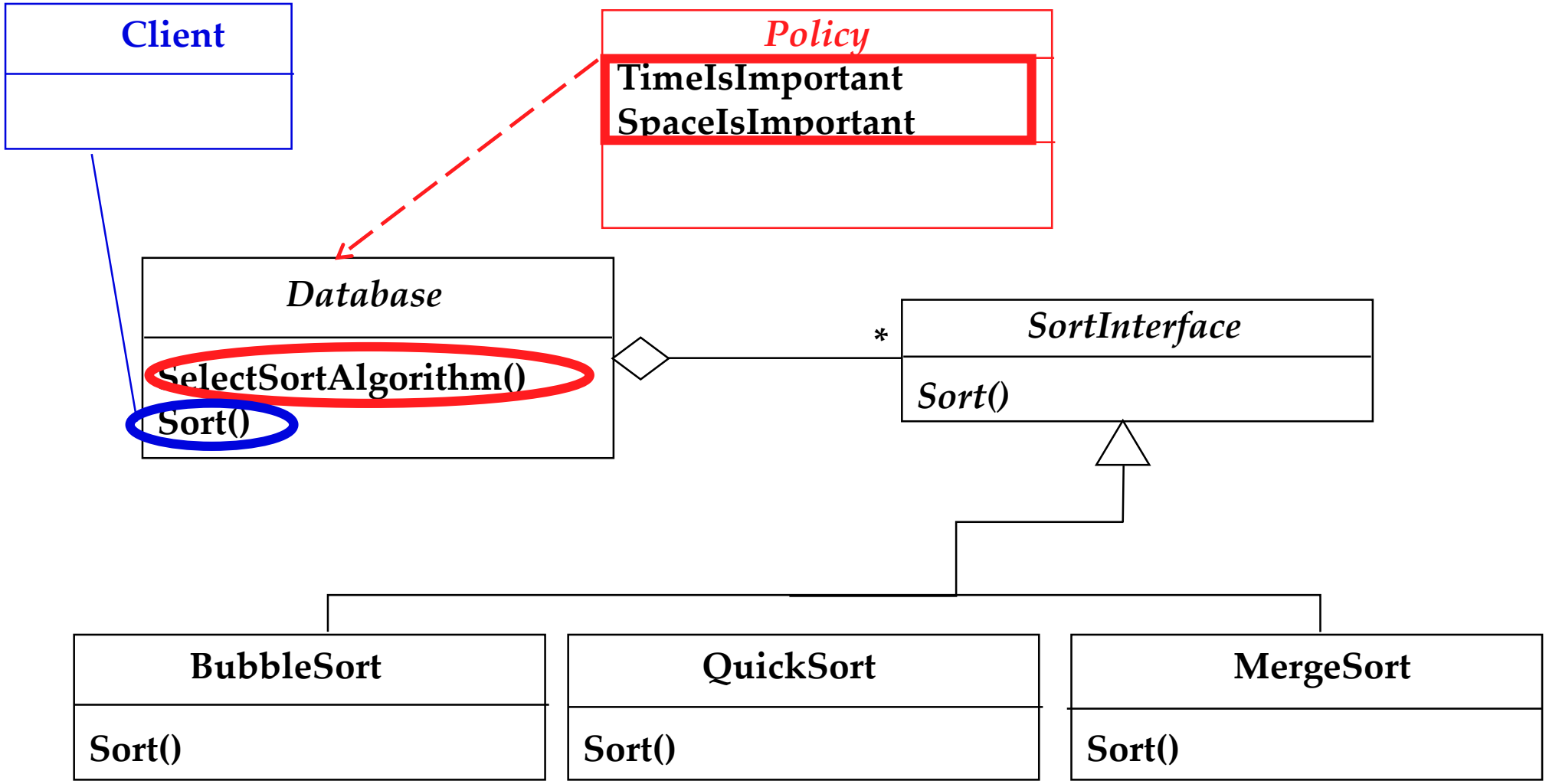
- Different algorithms exists for a specific task
  - We can switch between the algorithms at run time
- Examples of tasks:
  - Different collision strategies for objects in video games
  - Sorting a list of customers (Bubble sort, mergesort, quicksort)
- Different algorithms will be appropriate at different times
  - First build, testing the system, delivering the final product
- If we need a new algorithm, we can add it without disturbing the application or the other algorithms.

Strategy Pattern

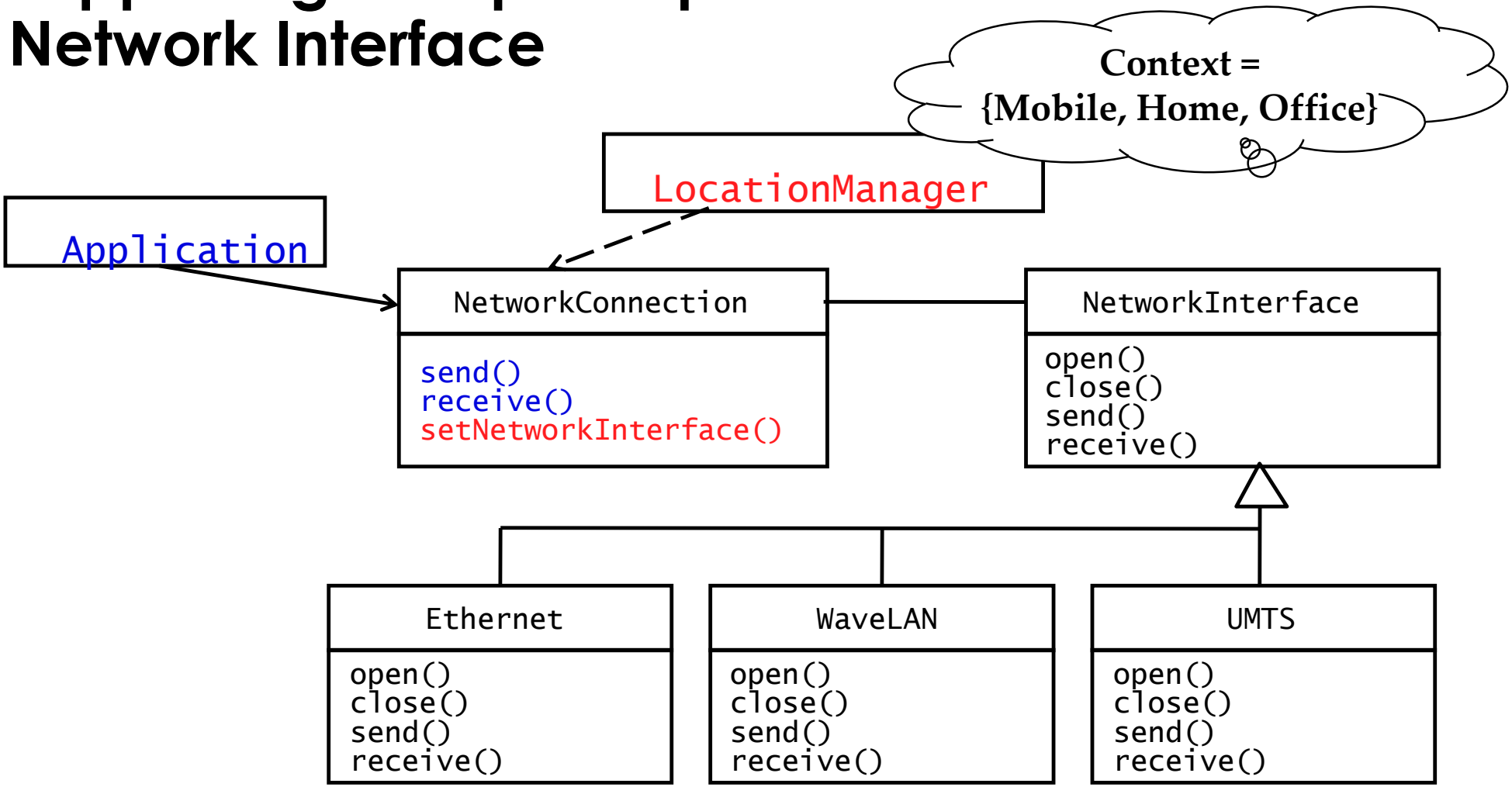


Policy decides which ConcreteStrategy is best in the current Context.

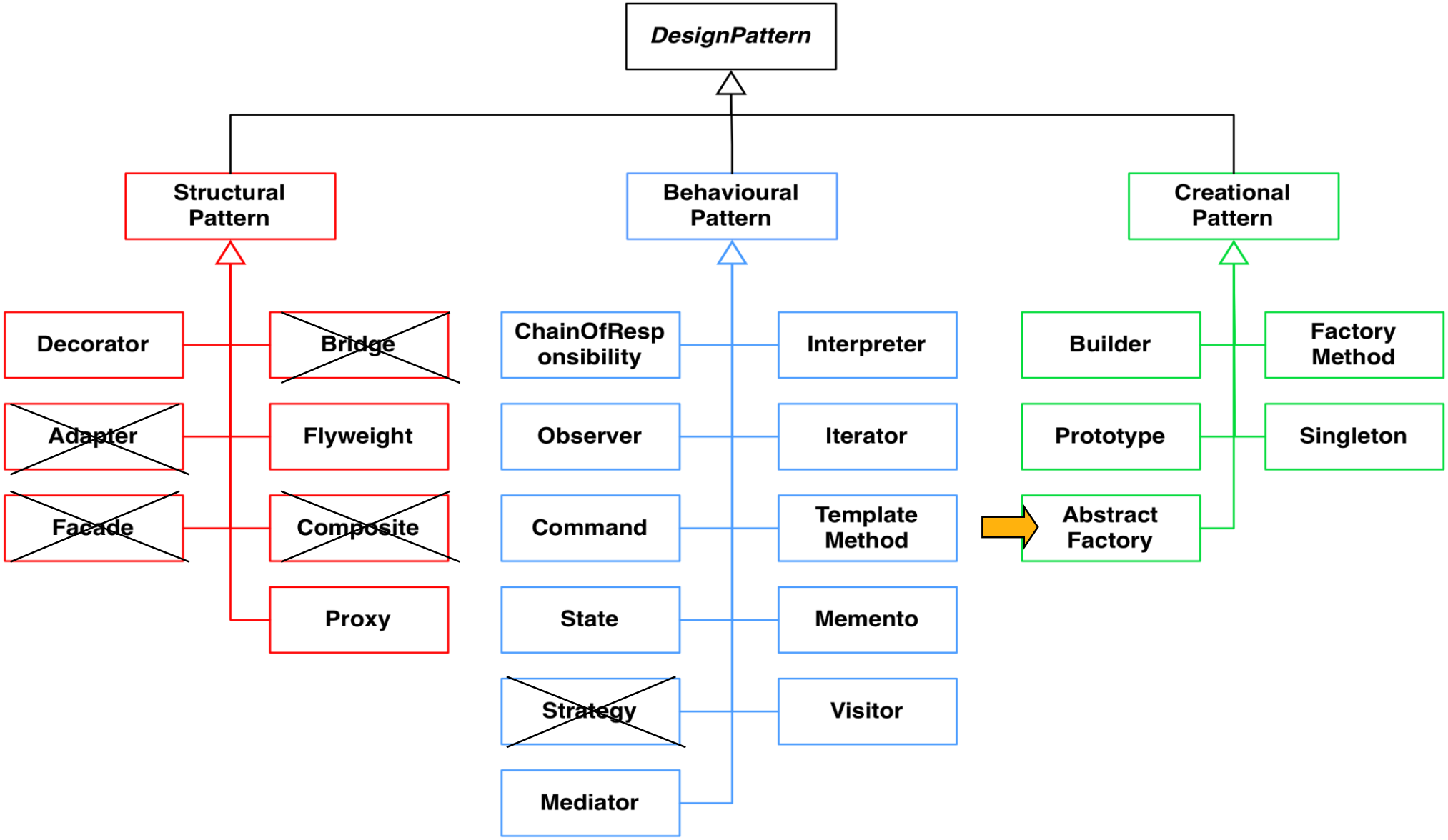
Using a Strategy Pattern to Decide between Algorithms at Runtime



Supporting Multiple implementations of a Network Interface



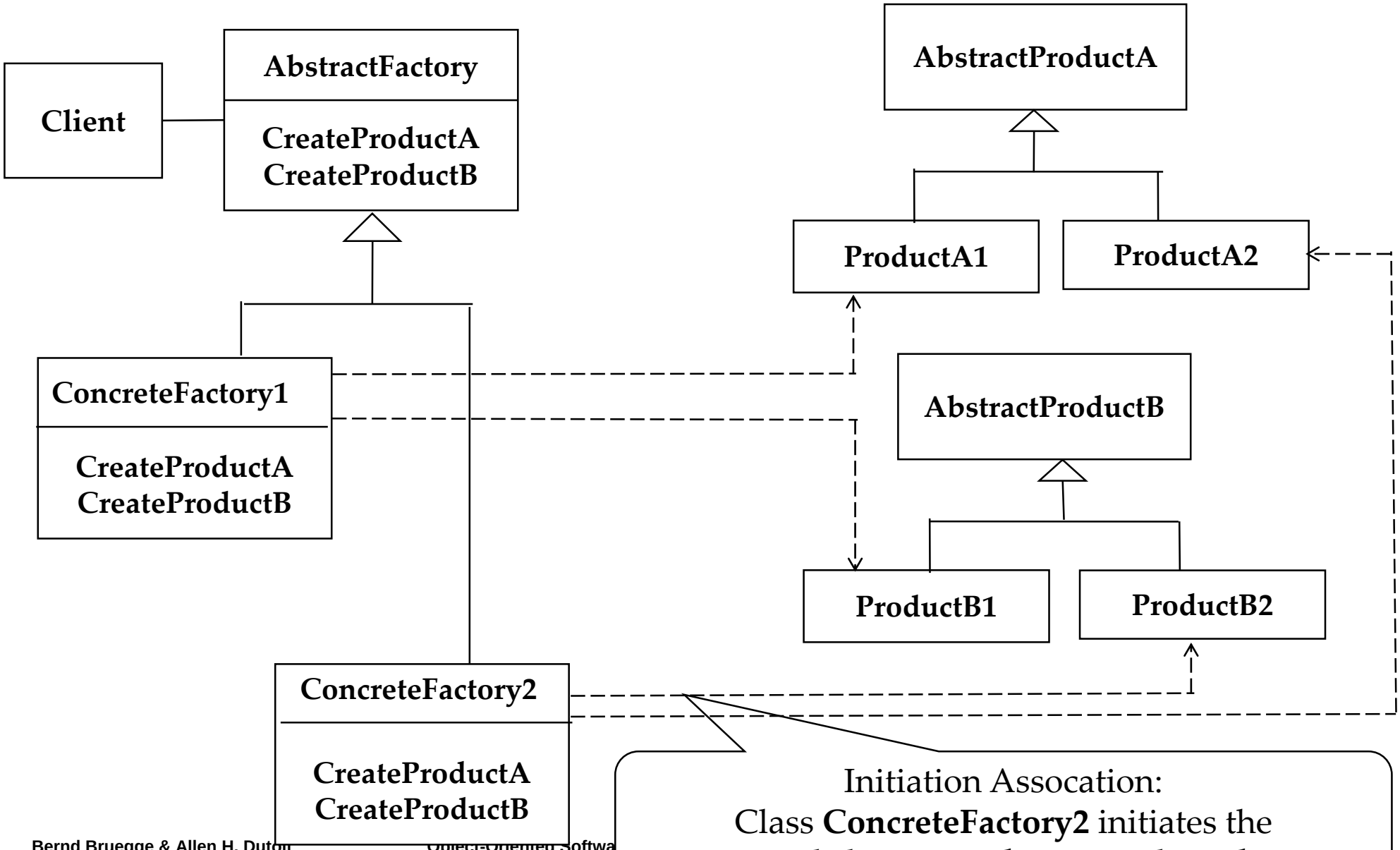
Taxonomy of Design Patterns



Abstract Factory Pattern Motivation

- Consider a user interface toolkit that supports multiple looks and feel standards for different operating systems:
  - How can you write a single user interface and make it portable across the different look and feel standards for these window managers?
- Consider a facility management system for an intelligent house that supports different control systems:
  - How can you write a single control system that is independent from the manufacturer?

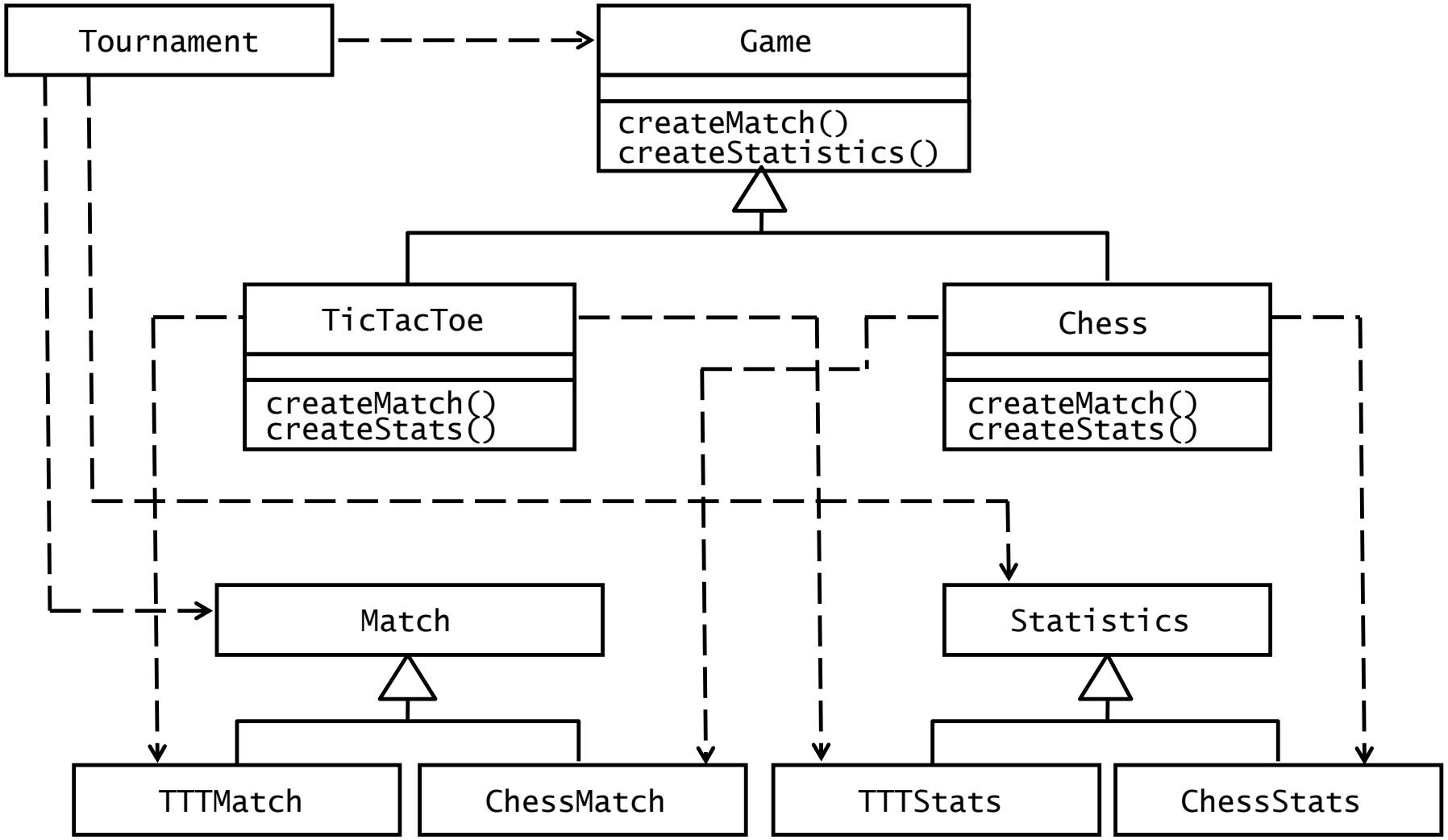
Abstract Factory



Applicability for Abstract Factory Pattern

- Independence from Initialization or Representation
- Manufacturer Independence
- Constraints on related products
  - A family of related products is designed to be used together
- Cope with upcoming change

Applying the Abstract Factory Pattern to Games



Clues in Nonfunctional Requirements for the Use of Design Patterns

- *Text:* "manufacturer independent",
  "device independent",
  "must support a family of products"

  => Abstract Factory Pattern
- *Text:* "must interface with an existing object"

  => Adapter Pattern
- *Text:* "must interface to several systems, some of them to be developed in the future",
  " an early prototype must be demonstrated"

  =>Bridge Pattern
- *Text:* "must interface to existing set of objects"

  => Façade Pattern

Clues in Nonfunctional Requirements for use of Design Patterns (2)

- *Text:* "complex structure",
  "must have variable depth and width"

  => Composite Pattern
- *Text:* "must provide a policy independent from the mechanism"

  => Strategy Pattern

Conclusion

Design patterns

- provide solutions to common problems
- lead to extensible models and code
- can be used as is or as examples of interface inheritance and delegation
- apply the same principles to structure and to behavior

- Design patterns solve a lot of your software development problems
  - Pattern-oriented development